

ADVANCEMENTS ON UML

Component / Deployment / Activity / Statechart Diagrams

Prof. Porfirio Tramontana

ptramont@unina.it

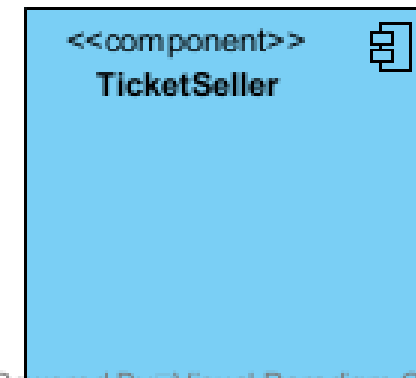
<https://www.docenti.unina.it/porfirio.tramontana>

Other Static Diagrams

Component Diagram and Deployment Diagram

Components

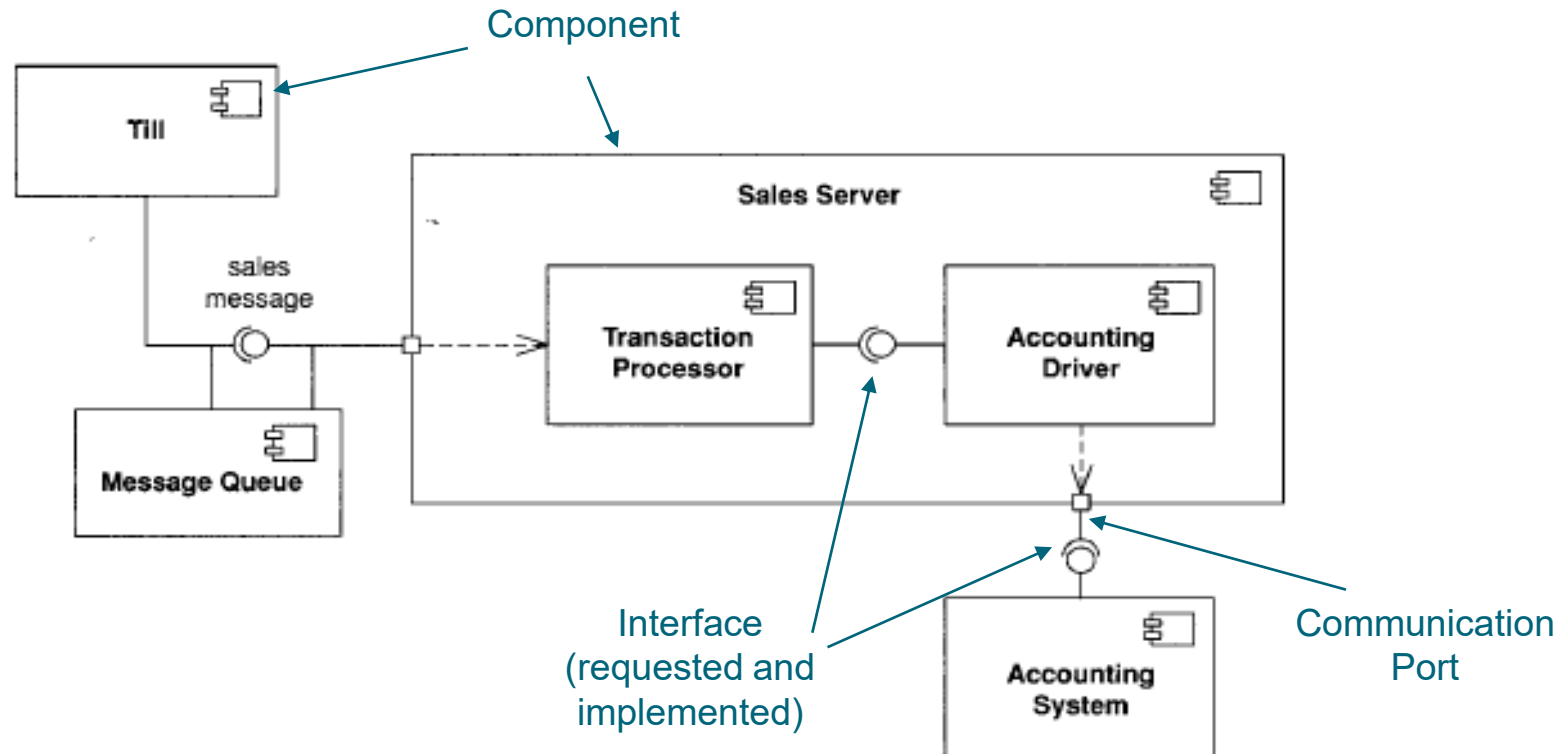
- A component is a container of other static UML elements, such as classes and packages
- Often, the component is identified as a physical component, such as a file containing source code, or an executable file, or a database, ...
- In other cases, the concept of component coincides with that of package (for example in java libraries "packaged" in a single file with an extension .jar)
- A component can also contain other components within it
 - The concept of **subsystem** is often used to intend a component including other components



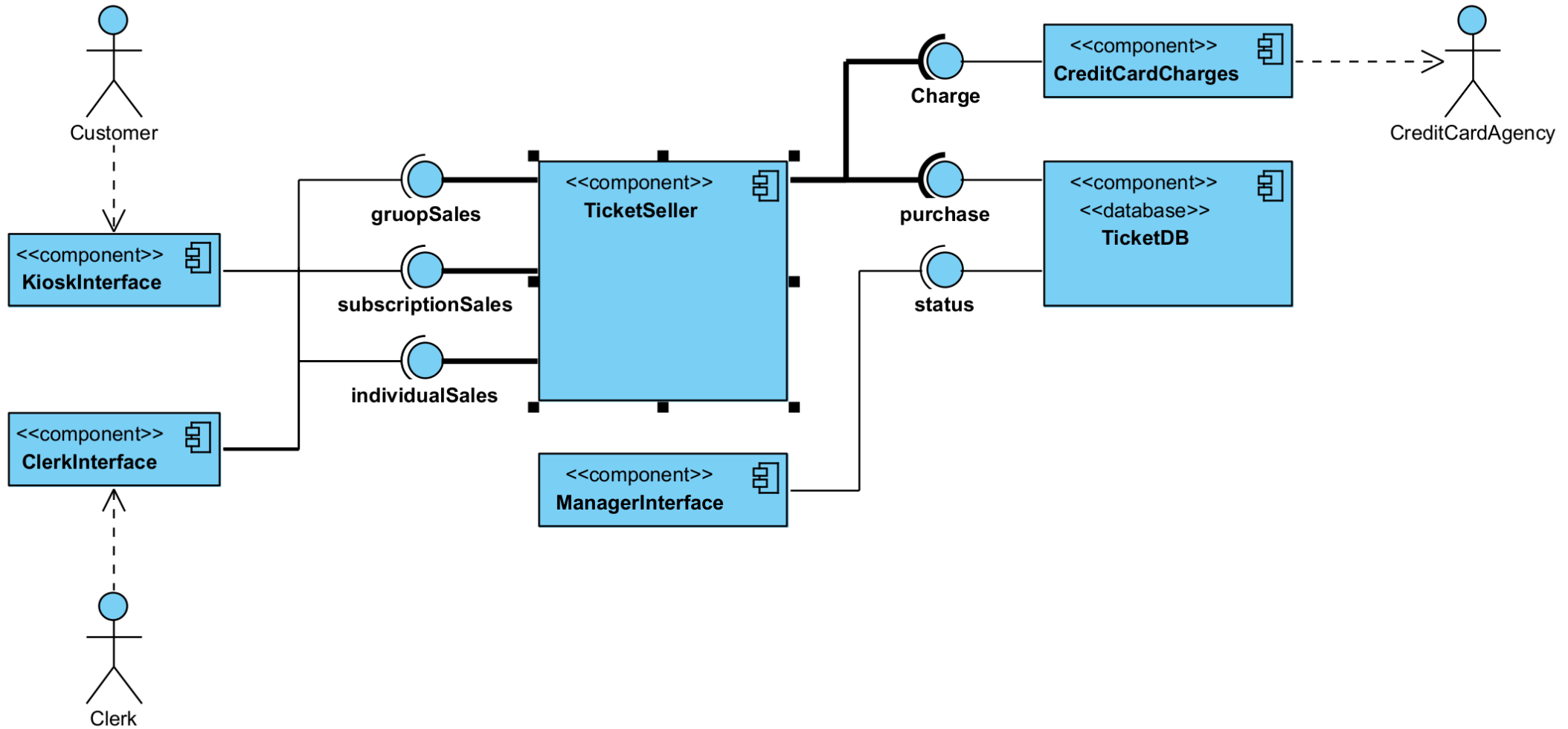
Component diagrams

- The component diagram shows sets of components and their relationships
- For each component, it is possible to explicitly show the interface that it exposes to the outside (using lollipop notation)
- The component diagram is, therefore, an architectural diagram that shows objects actually made, but which does not include information on the actual physical location of these components
- **Communication Ports** are little squares that express the need to transfer data between components, possibly with transformation/serialization/deserialization operations

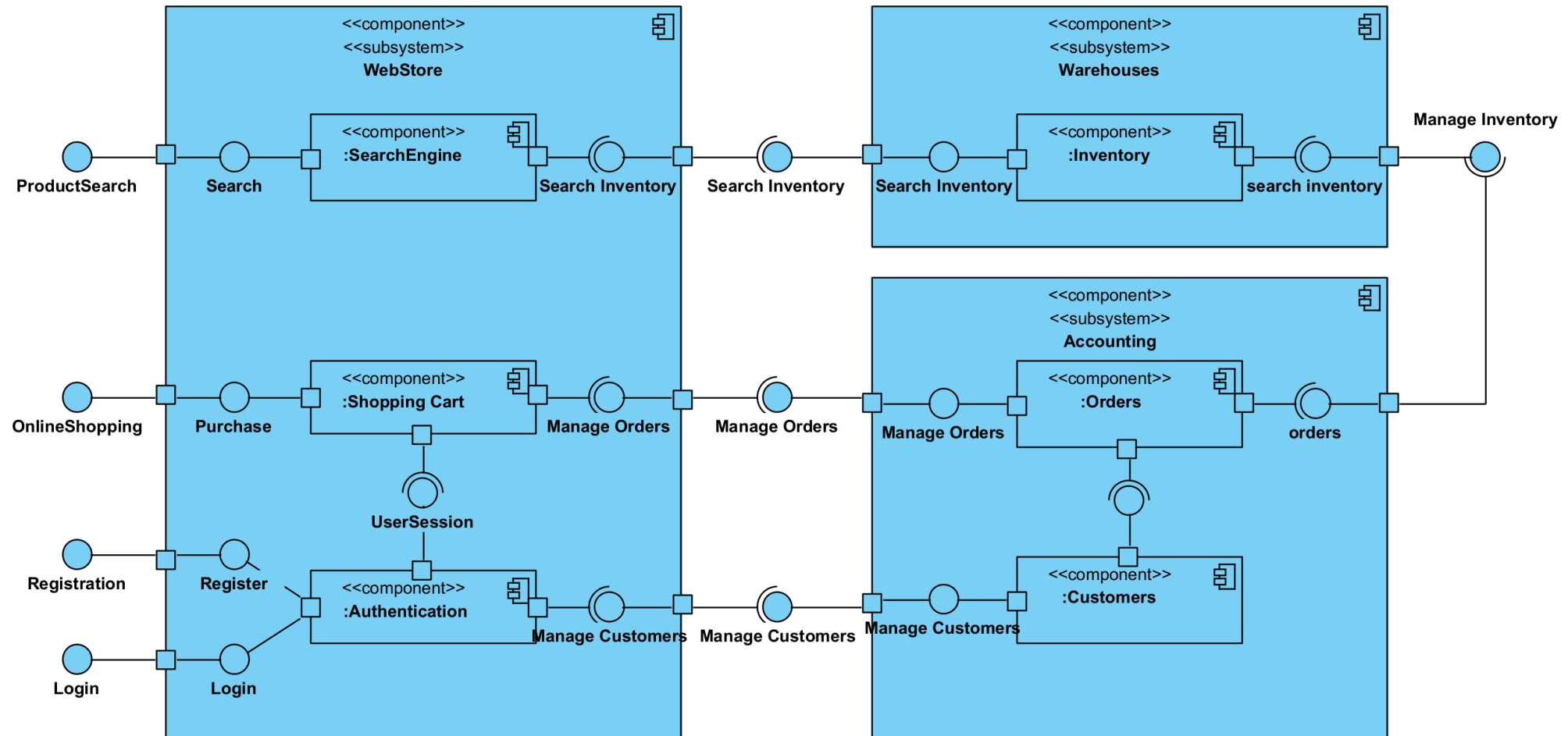
Example of Component Diagram



Example of Component Diagram

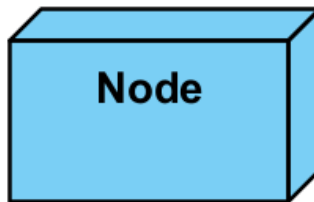


Example of Component Diagram with Subsystems



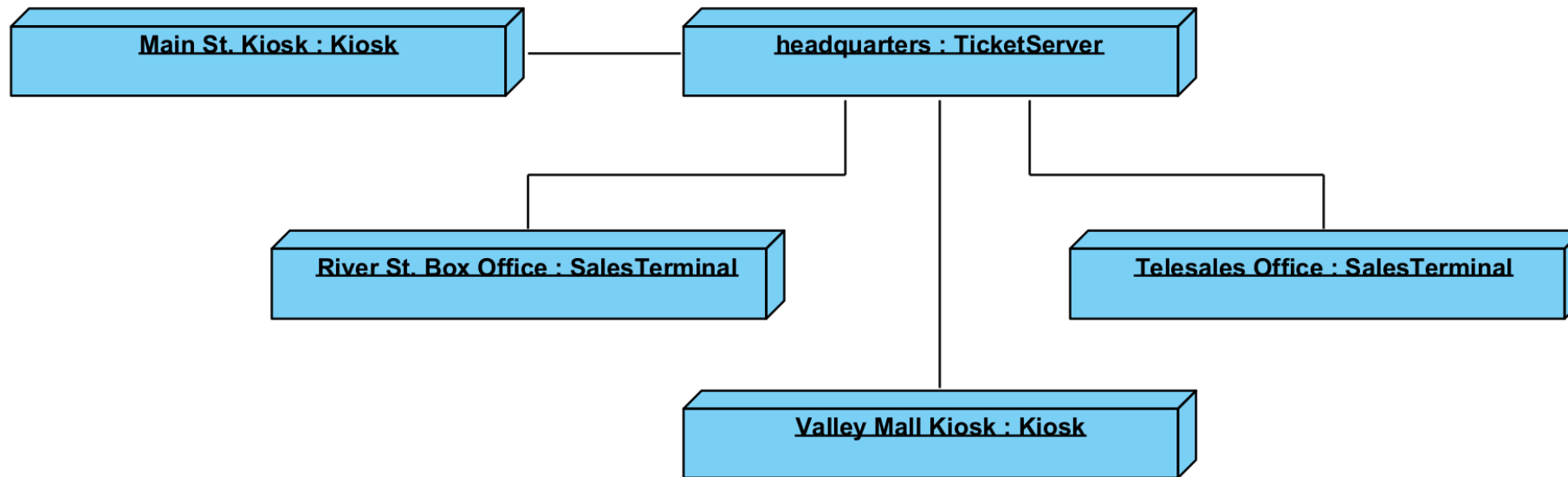
Deployment Diagram

- Deployment diagrams show the physical architecture of the system
- The fundamental element is the node, corresponding to a physical element of the system (e.g. a machine or a device)
- Components can be drawn inside the nodes, so that the deployment diagram can be linked to the corresponding component diagrams
- The products of processing are represented by artifacts
- The concept of artifact interprets any physical object produced by the system
- The arcs of a deployment diagram correspond to the physical connections between its nodes



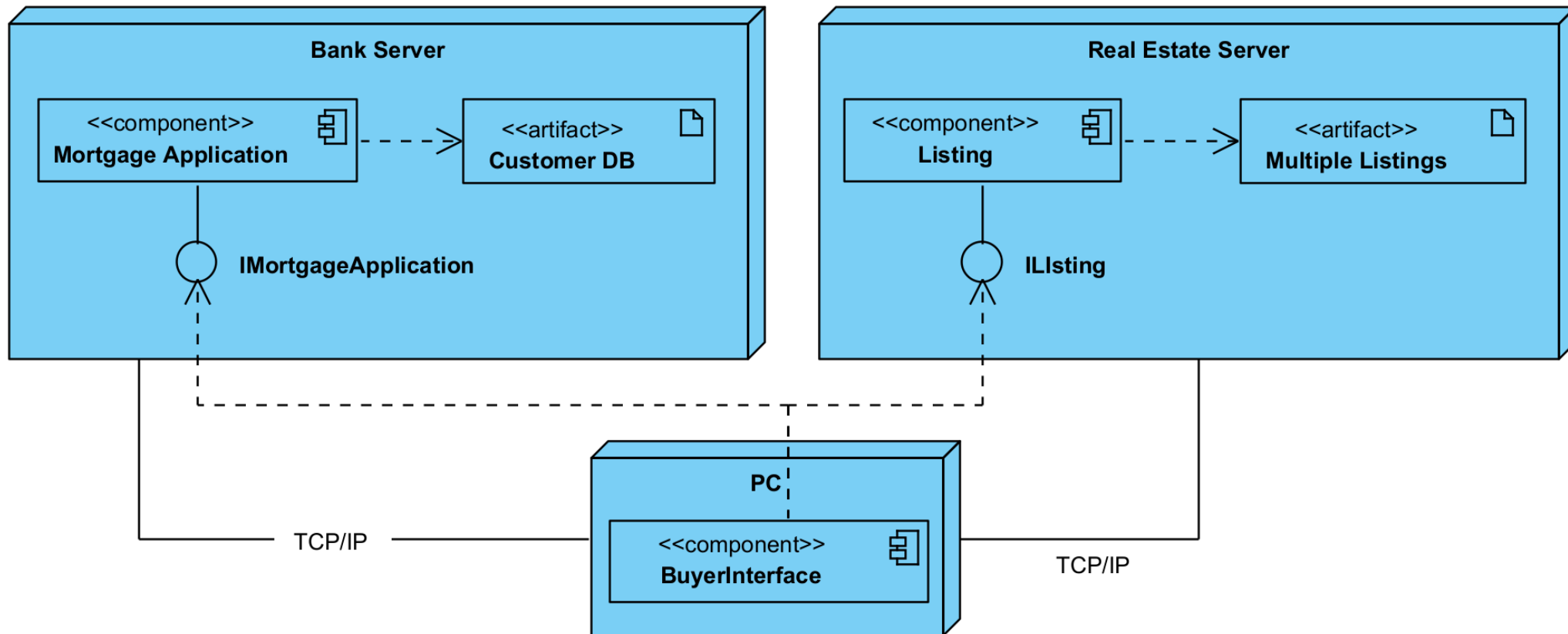
Example of Deployment Diagram

- On the left the name of the node instance, on the right the node type

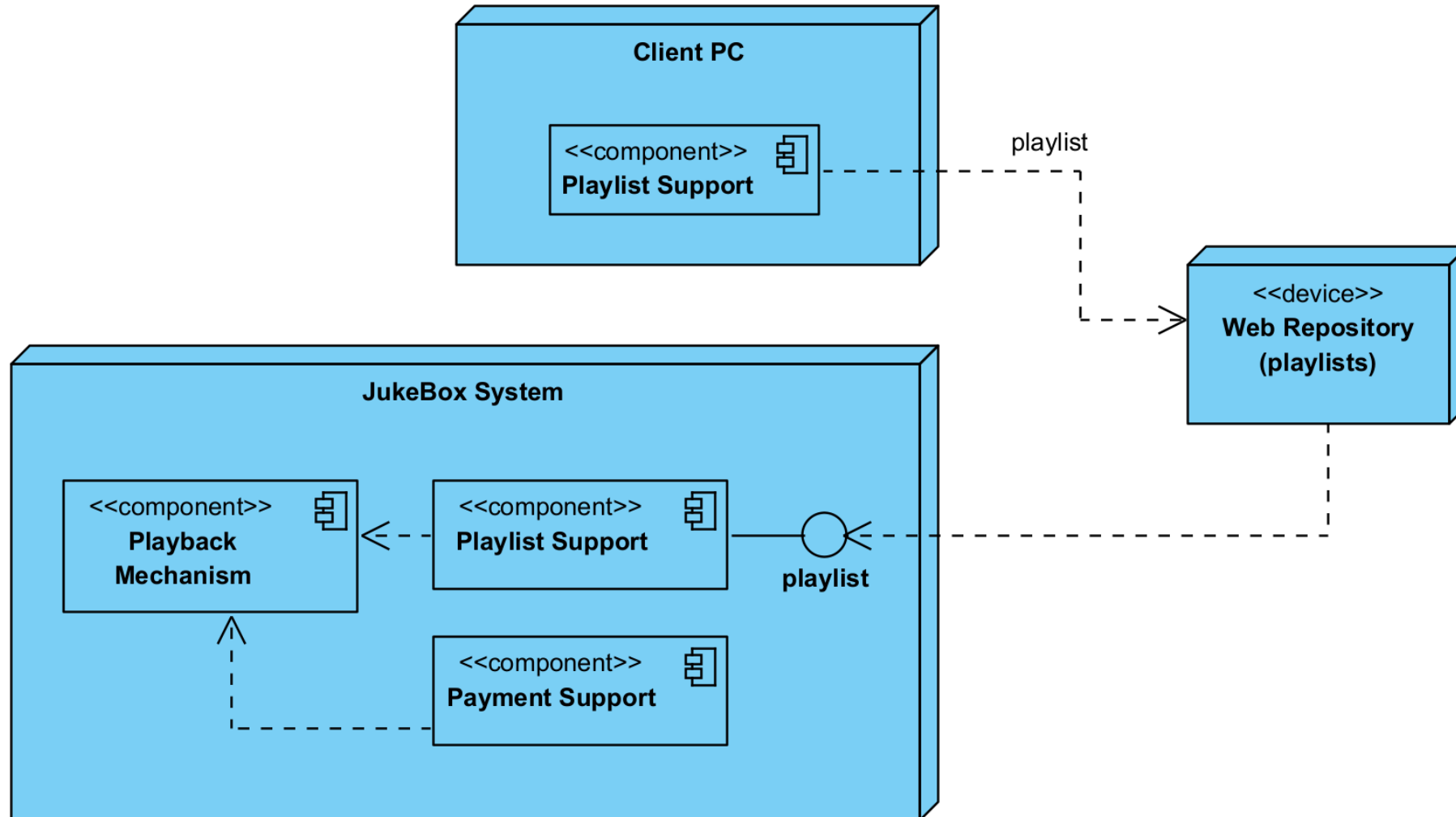


Deployment Diagram with Components

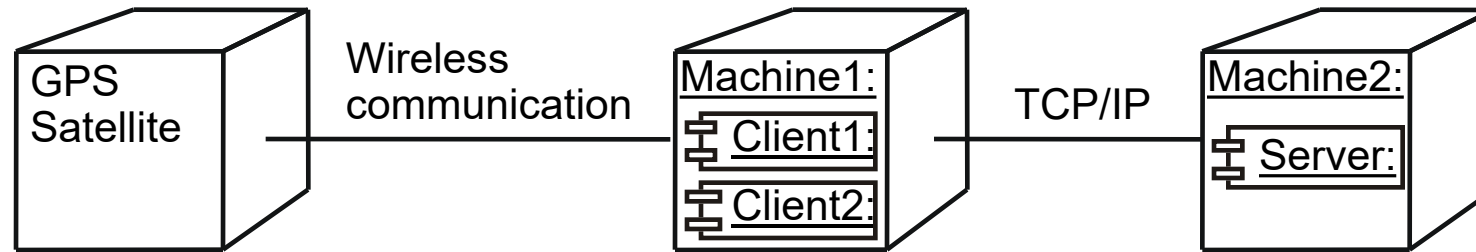
- Deployment Nodes include components



Example of Deployment Diagram



Example of Deployment Diagram



- Deployment diagrams are architectural diagrams that can be used in the architectural design phase and in the implementation phase
- Sometimes it is necessary to introduce them already in the analysis phase of the problem if one wants to represent architectural constraints placed at the basis of the problem

Behavioural Diagrams

Activity Diagrams

Statechart Diagrams

Riferimenti

- Martin Fowler, UML Distilled, capitoli
 - 10 (statechart diagram),
 - 11 (activity diagram),

UML Statecharts

Syntax and Semantics



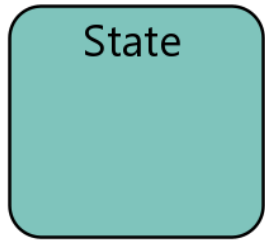
Statechart Diagrams

- A state diagram describes the behavior of a system, a part of a system, or a specific object.
 - At any given moment, the system or object is in a certain state.
 - Being in a state means that it will behave in a specific way in response to some event that will occur.
 - Some events can cause the system to change state.
 - In the new state, the system will behave differently in the face of events.
 - A state diagram is a directed graph where the nodes are the states and the edges are the transitions.

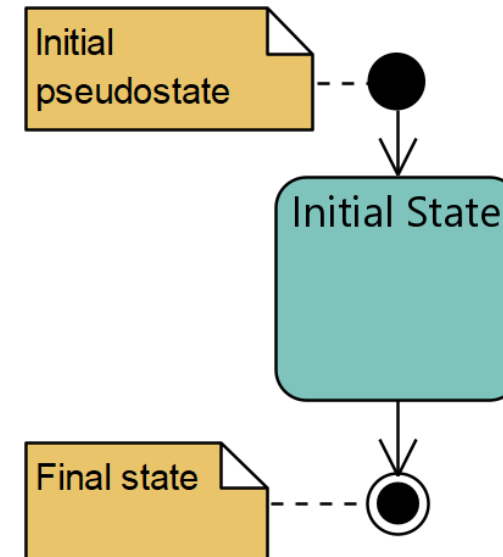
Regions, vertices and transitions

- A UML Statechart contains a top-level **region**
- A region contains **vertices** and **transitions**
- **Vertices** represent states
- **Transitions** are depicted as **directed edges** between two vertices
- Several kinds of vertices exist, with different semantics

Stati

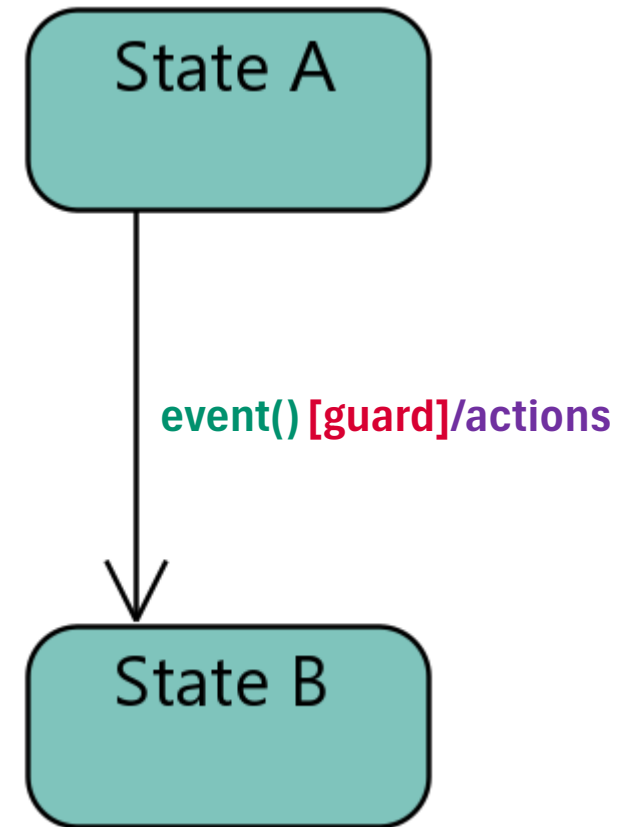


- A **state** is represented by a rounded rectangle with its name
 - At any given time, one and only one state of the diagram can occur
 - All states are considered stable: only one event can cause the transition from one state to another
- Special states (pseudostates):
 - The initial state (cannot be reached)
 - The final state (cannot be left)



Transitions: Syntax

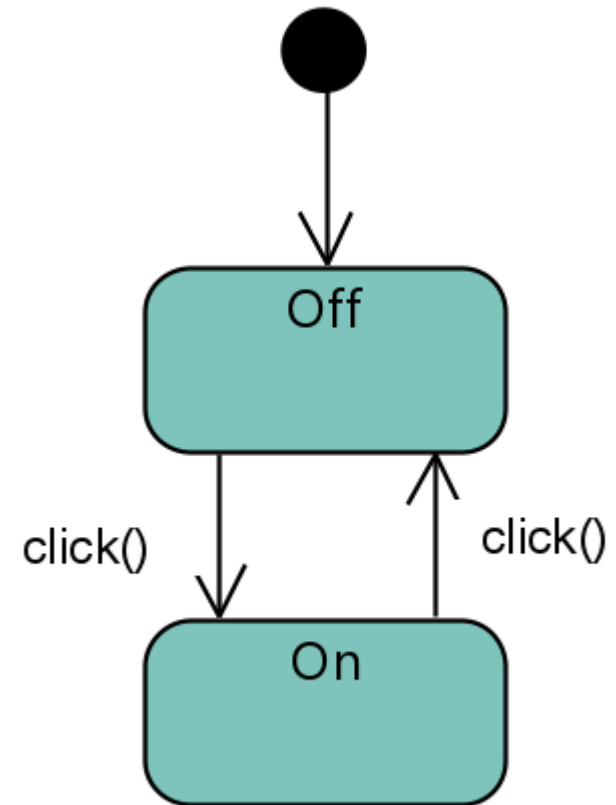
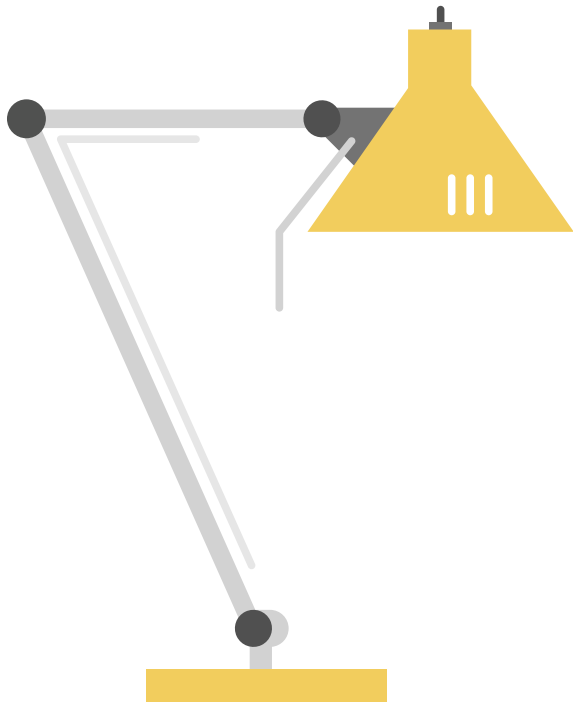
- Transitions indicate state changes
- Can be decorated with a label of the form:
triggers [guard] /actions
 - **triggers** is a list of events that may induce a state change
 - **guard** is a Boolean condition
 - **actions** is a list of operations to execute when the transition fires.
- All the above parts of the label are optional
- Self-transitions are possible



Transitions: Semantics

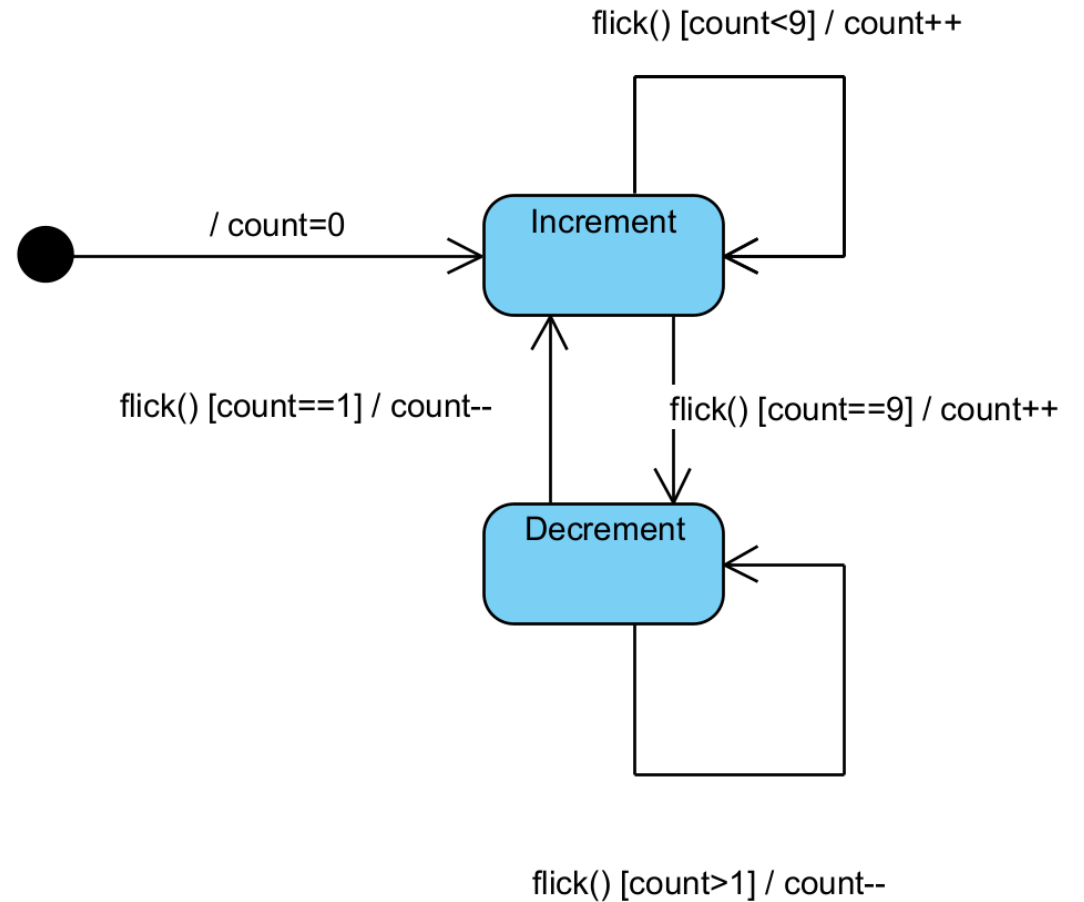
- For a transition to be **fireable**:
 - Events matching all of the triggers should be fired;
 - The condition in the guard must evaluate to TRUE.
- A **spontaneous** transition is one with no **triggers** and no **guard**.
- After a transition fires, its associated list of actions is executed.
- If multiple transitions are fireable, only one of them actually fires (nondeterministically determined).

Example: a lamp with a single button

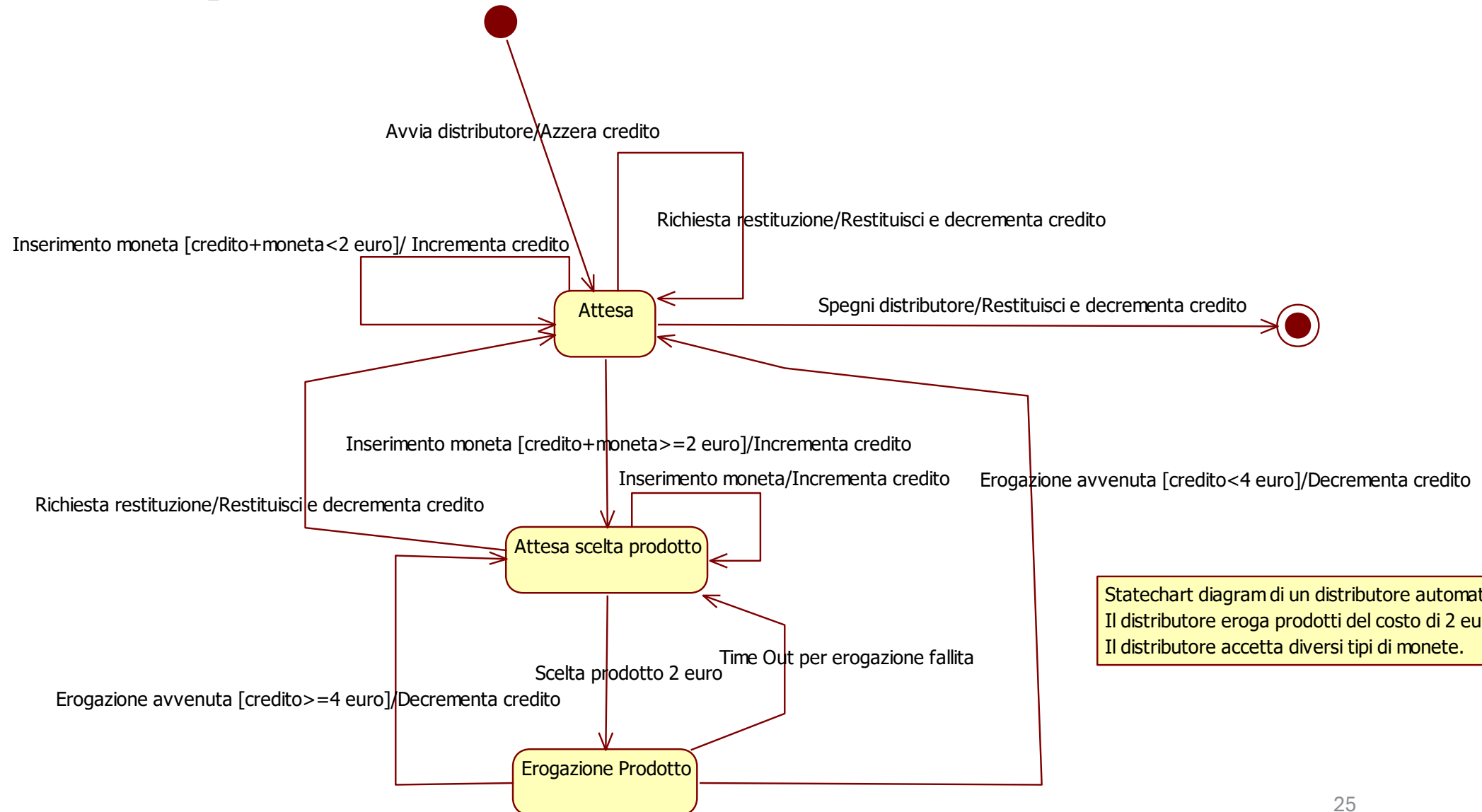


Example: a simple counter (Java)

```
public class Counter {  
    private int count = 0;  
    private String mode = "increment";  
    public void flick() {  
  
        if(mode.equals("increment"))  
            count++;  
        else  
            count--;  
  
        if(count==10)  
            mode = "decrement";  
        else if (count==0)  
            mode = "increment";  
  
    }  
}
```



Example : Coffee Machine



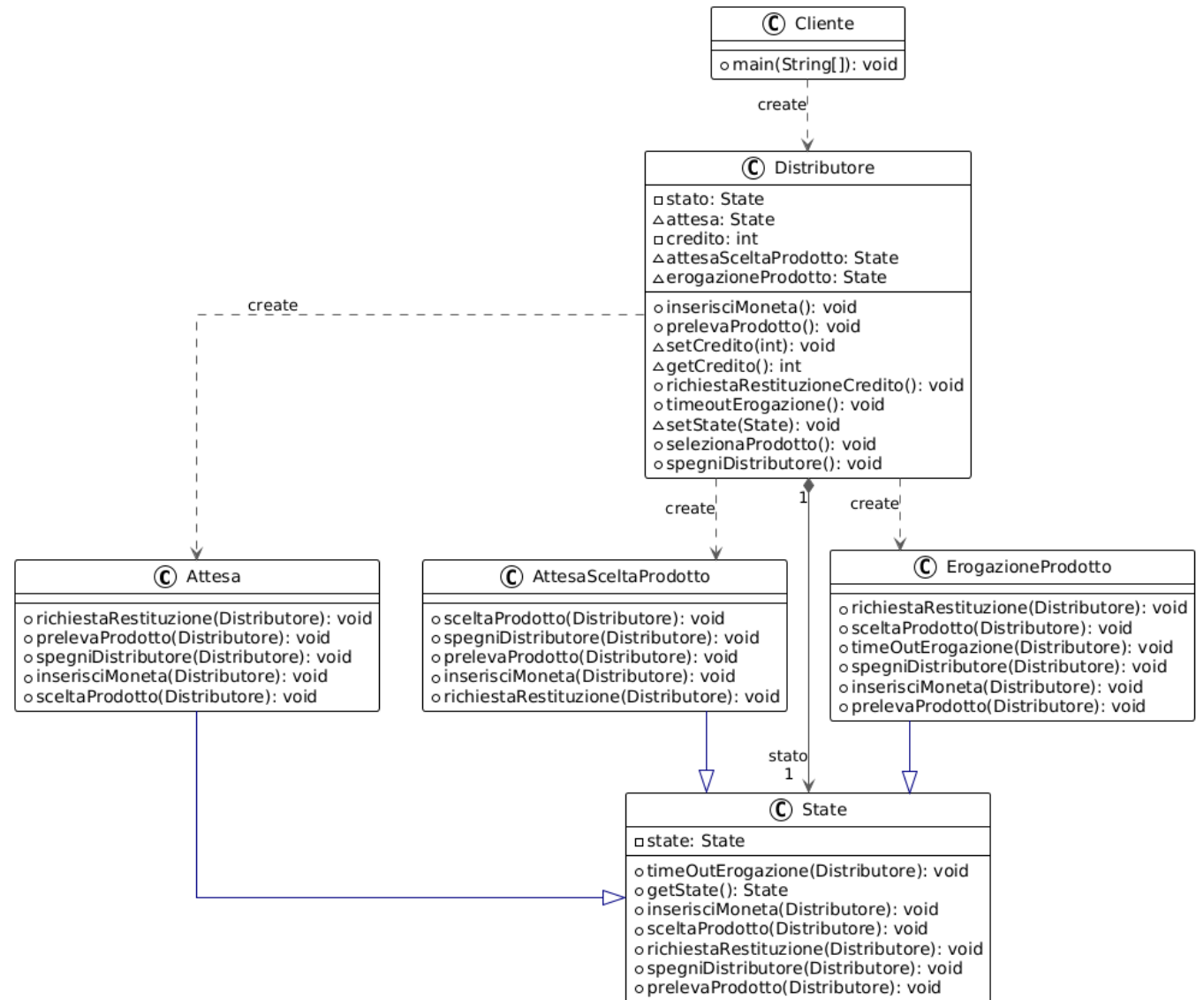
Statechart implementation

- A detail-level statechart can be implemented using a chain of nested switches

```
Switch (statoCorrente) {  
    Case statol:  
        Switch (unEvento)  
            Case eventol:  
                if (guardial)  
                    azioneEventol;  
                Break;  
    Case stato2 ...  
    ...  
}
```


Pattern State

- A more effective solution in object-oriented systems is to use the pattern state
- The statechart is translated as:
 - A general class that declares the methods corresponding to all actions but implements them without doing anything
 - A series of derived classes, corresponding to the states of the diagram, which implement only the methods corresponding to the events that can be triggered on that state (by overriding)



The code of this solution is on Teams

Example of Pattern State

Distributore Class (container)

```
public class Distributore {
    private State stato;

    static State attesaSceltaProdotto = new AttesaSceltaProdotto();
    static State attesa = new Attesa();
    static State erogazioneProdotto = new ErogazioneProdotto();
    private int credito=0;

    public Distributore() {setState(attesa); }
    void setState (State newStato) {    this.stato=newStato;  }

    void setCredito(int credito) {this.credito=credito;}
    int getCredito() { return credito; }

    public void inserisciMoneta() {    stato.inserisciMoneta(this);  }
    public void selezionaProdotto() {    stato.sceltaProdotto(this);  }

    public void richiestaRestituzioneCredito()
    {    stato.richiestaRestituzione(this);  }

    public void spegniDistributore() {    stato.spegniDistributore(this);  }
    public void prelevaProdotto() {    stato.prelevaProdotto(this);  }
    public void timeoutErogazione() { stato.timeOutErogazione(this); }
}
```

Generic Class for States

```
public class State {
    private State state;

    public State getState() {
        return state;
    }

    public void inserisciMoneta(Distributore distributore) {}
    public void richiestaRestituzione(Distributore distributore) {}
    public void sceltaProdotto(Distributore distributore) {}
    public void spegniDistributore(Distributore distributore) {}
    public void prelevaProdotto(Distributore distributore) {}
    public void timeOutErogazione(Distributore distributore){}
}
```

Fragments of code

State Attesa Class

- public class Attesa extends State {
 @Override
 public void inserisciMoneta(Distributore distributore) {
 distributore.setCredito(distributore.getCredito()+1);
 System.out.println("Credito attuale : "+distributore.getCredito());
 if (distributore.getCredito()>=2){
 distributore.setState(Distributore.attesaSceltaProdotto);
 }
 }

 @Override
 public void richiestaRestituzione(Distributore distributore) {
 System.out.println("Ritira il tuo credito : "+distributore.getCredito());
 distributore.setCredito(0);
 }

 @Override
 public void sceltaProdotto(Distributore distributore) {
 System.out.println("Non hai credito sufficiente");
 }

 @Override
 public void spegniDistributore(Distributore distributore) {
 System.out.println("Ritira il tuo credito : "+distributore.getCredito());
 distributore.setCredito(0);
 System.out.println("Spendo il distributore");
 System.exit(0);
 }

 @Override
 public void prelevaProdotto(Distributore distributore) {
 System.out.println("Il prodotto non è pronto");
 }
}

Example Main (for testing)

```
public class Cliente {  
    public static void main(String[] args) {  
        Distributore dis = new Distributore();  
  
        System.out.print("Scelgo un prodotto. "); dis.selezionaProdotto();  
        //selezioniamo un prodotto senza prima inserire una moneta  
        System.out.print("Inserisco un euro. "); dis.inserisciMoneta();  
        System.out.print("Scelgo un prodotto. ");  
        dis.selezionaProdotto(); //selezioniamo un prodotto  
        System.out.print("Inserisco un euro. "); dis.inserisciMoneta();  
        System.out.print("Scelgo un prodotto. "); dis.selezionaProdotto();  
        //seleziona il prodotto  
        System.out.print("Prelevo il prodotto. "); dis.prelevaProdotto();  
        System.out.print("Inserisco un euro. "); dis.inserisciMoneta();  
        System.out.print("Inserisco un euro. "); dis.inserisciMoneta();  
        System.out.print("Inserisco un euro. "); dis.inserisciMoneta();  
        System.out.print("Inserisco un euro. "); dis.inserisciMoneta();  
        System.out.print("Scelgo un prodotto. "); dis.selezionaProdotto();  
        System.out.print("Scelgo un prodotto. "); dis.selezionaProdotto();  
        System.out.print("Richiedo la restituzione del credito. ");  
        dis.richiestaRestituzioneCredito();  
        System.out.print("Inserisco un euro. "); dis.inserisciMoneta();  
        System.out.print("Prelevo il prodotto. "); dis.prelevaProdotto();  
        System.out.print("Spendo il distributore. "); dis.spegniDistributore();  
        System.out.print("Inserisco un euro. "); dis.inserisciMoneta();  
    }  
}
```

Polymorphism is used to adapt the behavior with respect to different states

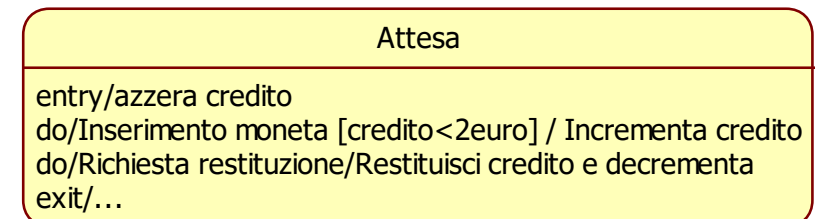
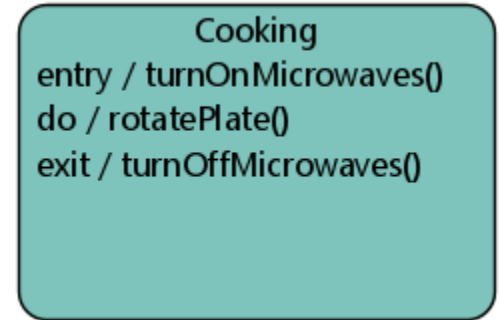
States: internal activities

States can (optionally) contain a list of **internal activities**.

Each activity is characterized by a **label** indicating **when** the activity is to be invoked.

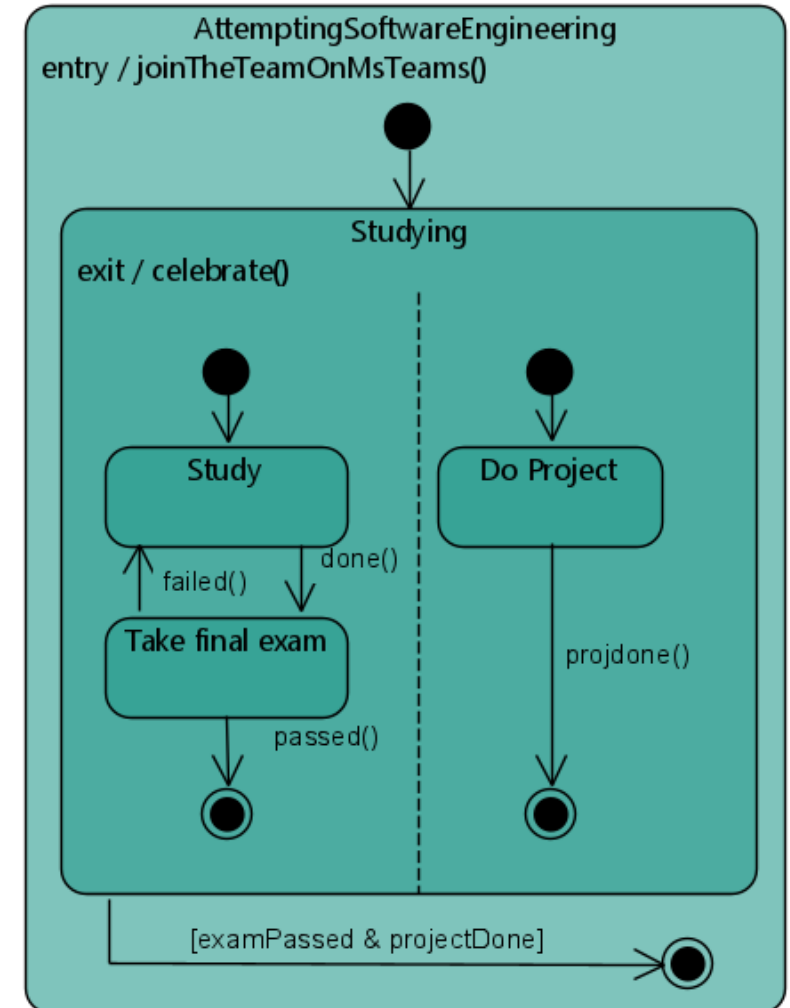
Reserved labels:

- **entry** / activity performed upon entry
- **do** / performed as long as the system is in the state (after entry activities completed)
- **exit** / activity performed upon exit

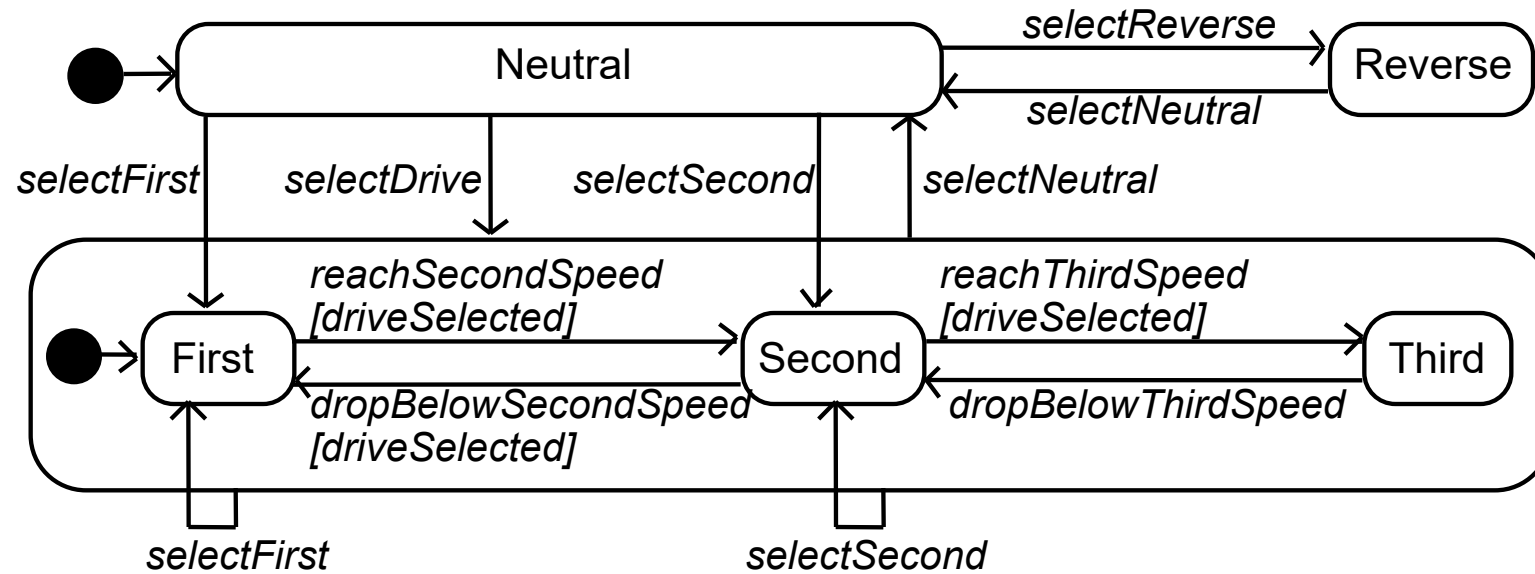


Composite States: parallel regions

- Composite states include statechart diagrams inside single states
- Composite states can contain multiple regions, representing behaviours that may occur in parallel
- When exiting from a composite state, all of its regions are terminated
- Analogously, there are **Activity States**, where inside a state an Activity Diagram is depicted



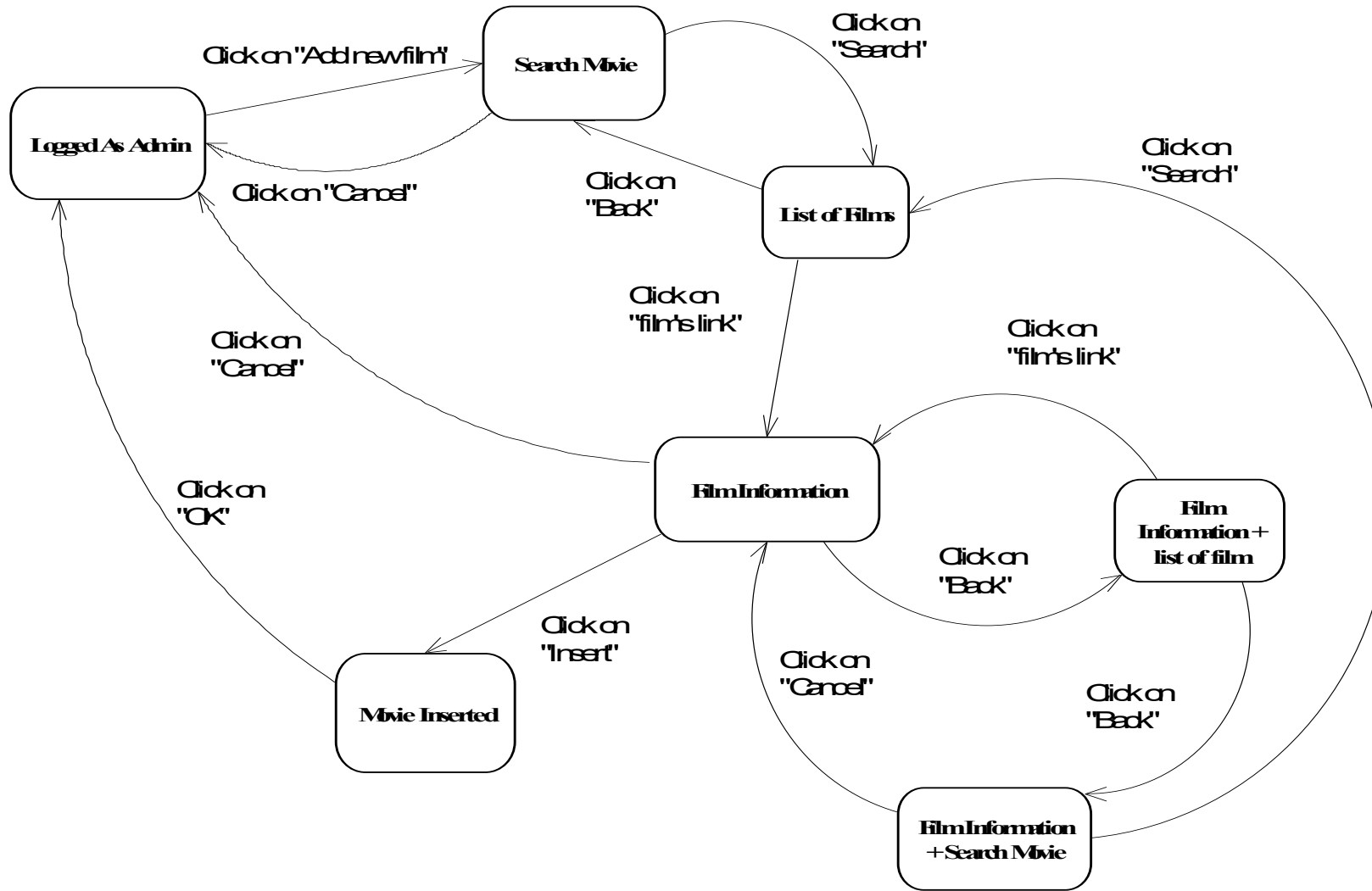
Example of Composite State



Statechart diagram and graphical user interface

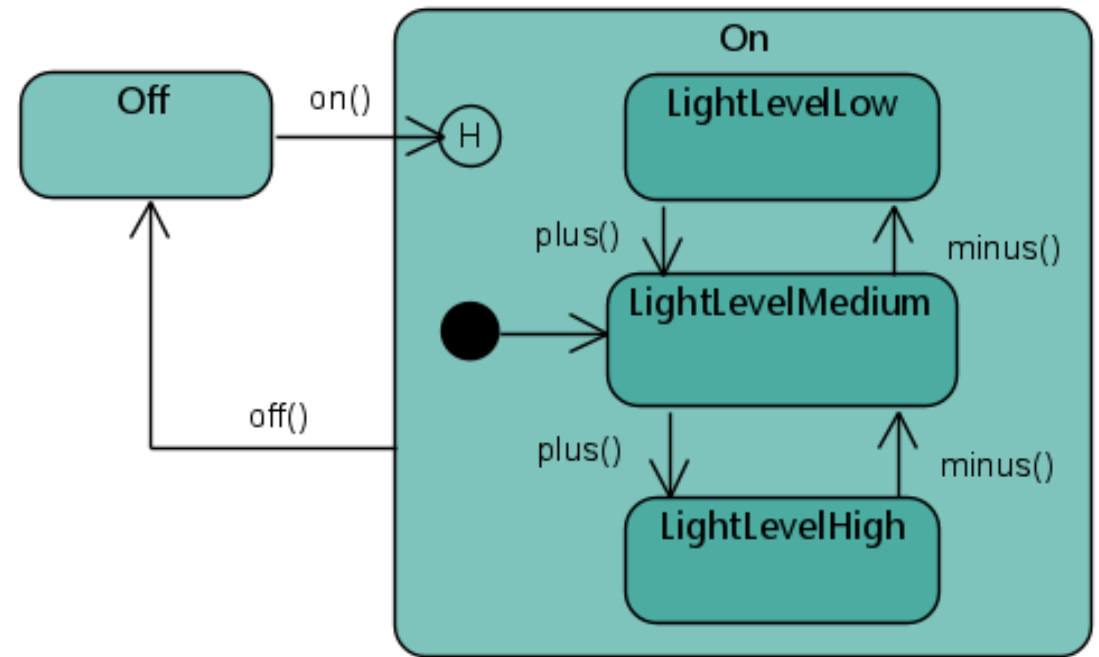
- Statechart diagrams are particularly useful in modeling event-driven systems
 - A very important special case is represented by visual user interfaces (GUI – Graphical User Interface)
 - In this case, a state can correspond to a window, while a trigger event corresponds to a user-generated event (a keystroke, a mouse click, etc.).

Example - FSM model of a UI



Shallow History Pseudostates

- Depicted as a $\odot$ H
- Represents the most recently active state of a composite state, but not substates of that substate!
- Only in composite states, and only one per region



Statechart for a lamp with three different light levels

Activity Diagrams

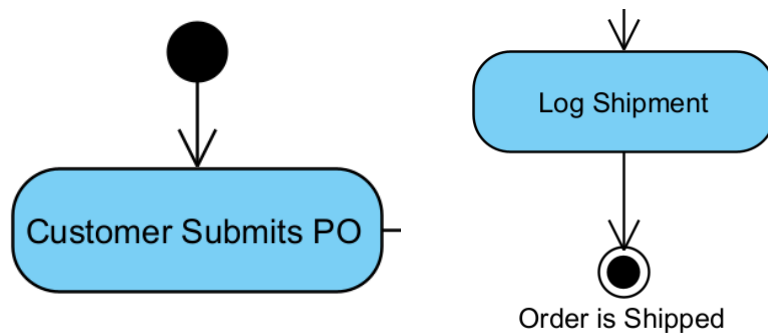


Activity Diagrams

- An activity diagram
 - Can be used to model a **workflow**
 - In the analysis phase, it can be used to model a **real problem**
 - At the high-level design time it can be used to model the interaction between conceptual objects in solving a problem
 - In the detailed design phase it can be used to model an algorithm, possibly concurrent, which must be implemented
- An activity diagram is also an extension of a flowchart, with the introduction, in particular, of concurrent executions
 - An activity diagram was born, in the context of UML, as an extension of the statechart diagram
 - In the latest versions, however, it has been completely detached from it, so it makes sense to study it separately

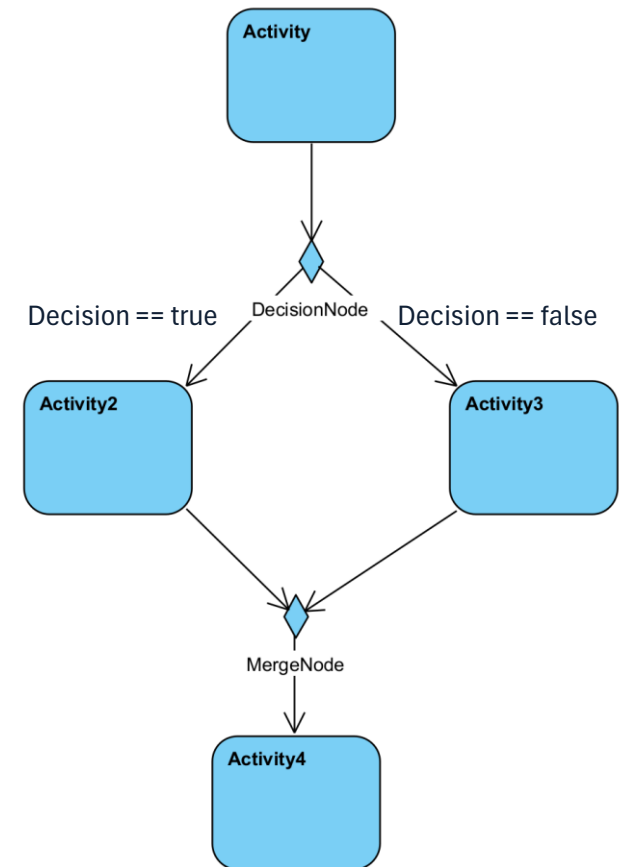
Activities and flows

- **Activities** are defined as processing actions that the system must perform and are modeled with rounded rectangles identified by the name of the activity itself
- The flow between activities is modeled with incoming/outgoing arrows from the task
- By convention, **entry** into the activity coincides with its beginning; **Exiting** the activity coincides with its completion
- Here too there are **initial** and **final** states delimiting the diagram



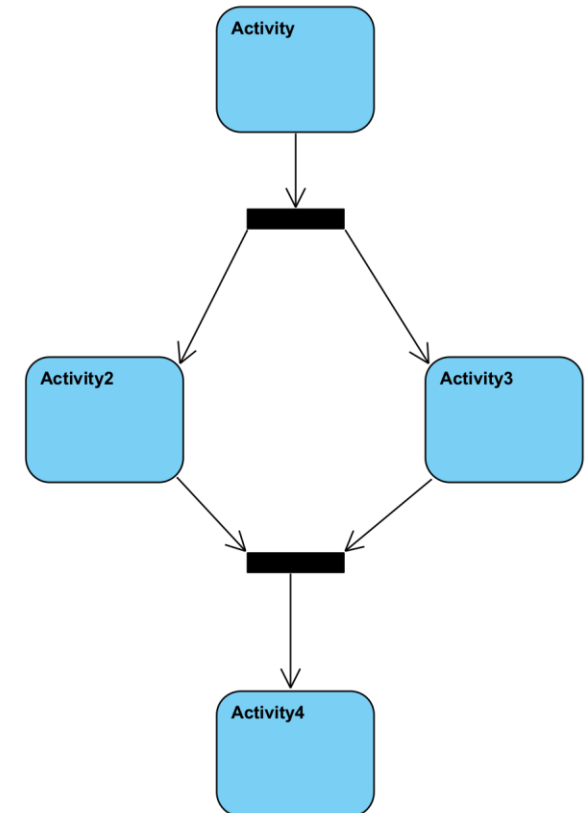
Branch

- The branch is a decision activity
 - Corresponds to an IF
 - It is modeled with a rhombus that has only one incoming arch and more than one outgoing arch
 - Each outgoing arch is labeled with a guard condition (in square brackets)
 - The conditions must be mutually exclusive to each other
- The merge is the dual element of the branch
 - Corresponds to an END IF
 - It is also modeled with a rhombus, but with only one outgoing arch and more than one incoming arch

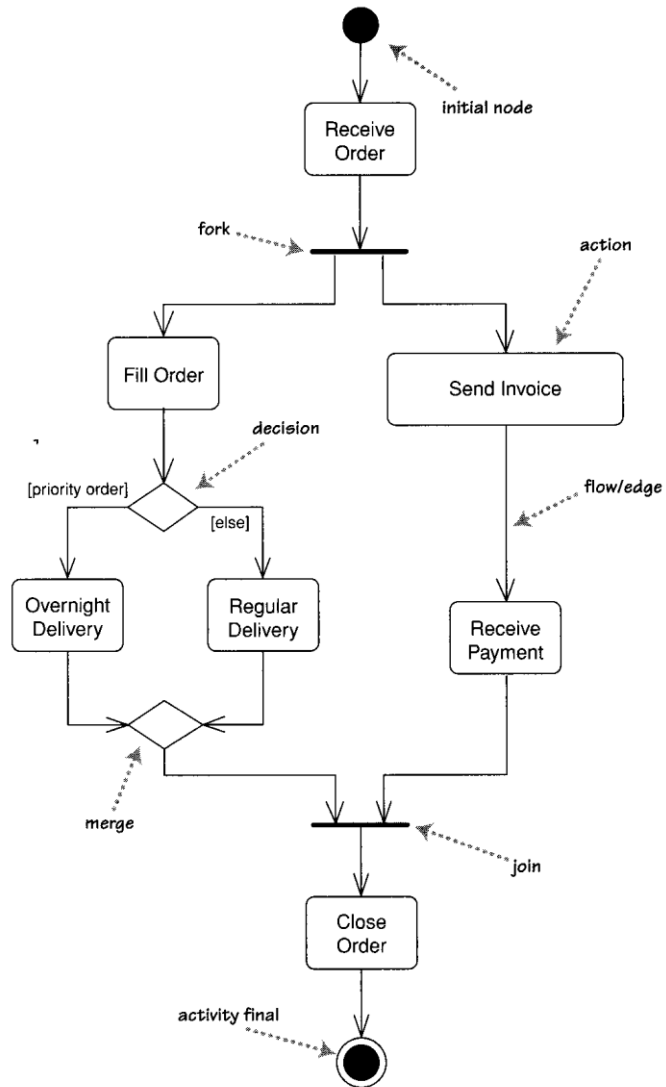


Fork and Join

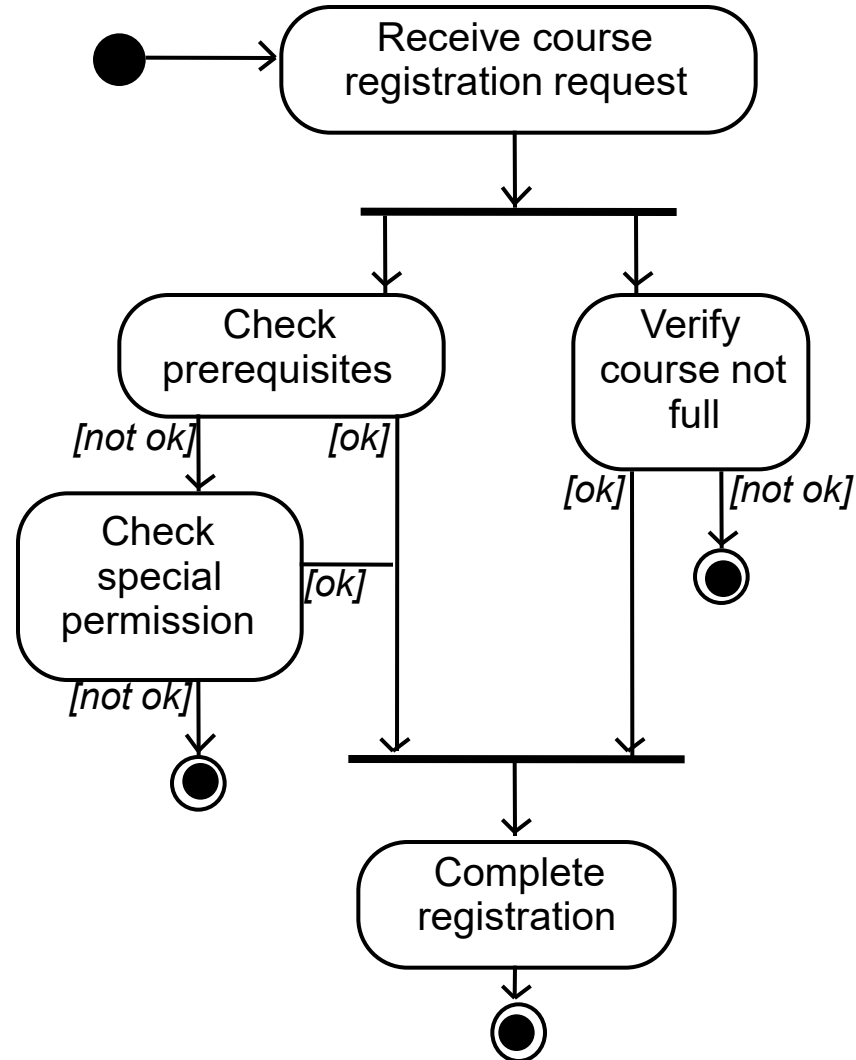
- The fork represents a separation of the flow in two or more threads, that can act concurrently
 - It is represented by a horizontal bar in which only one arc enters and more than one comes out
 - The outgoing arcs are not labeled, and it is understood that as many threads are started at the same time as a result of the bifurcation
- Join is the dual of the fork
 - It is also represented by a horizontal bar in which more than one arc enters and only one comes out
 - Forks and Joins correspond exactly to the conceptual primitives of concurrent programming of the same name



Activity diagrams: example

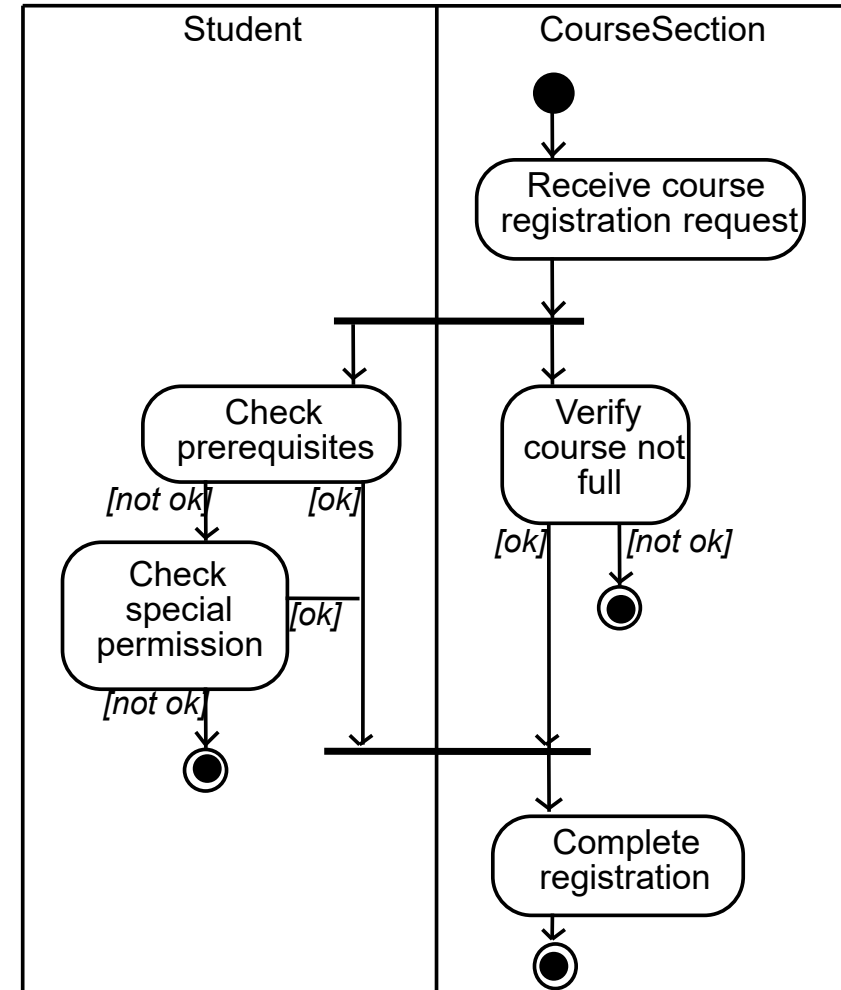


Activity diagrams – example

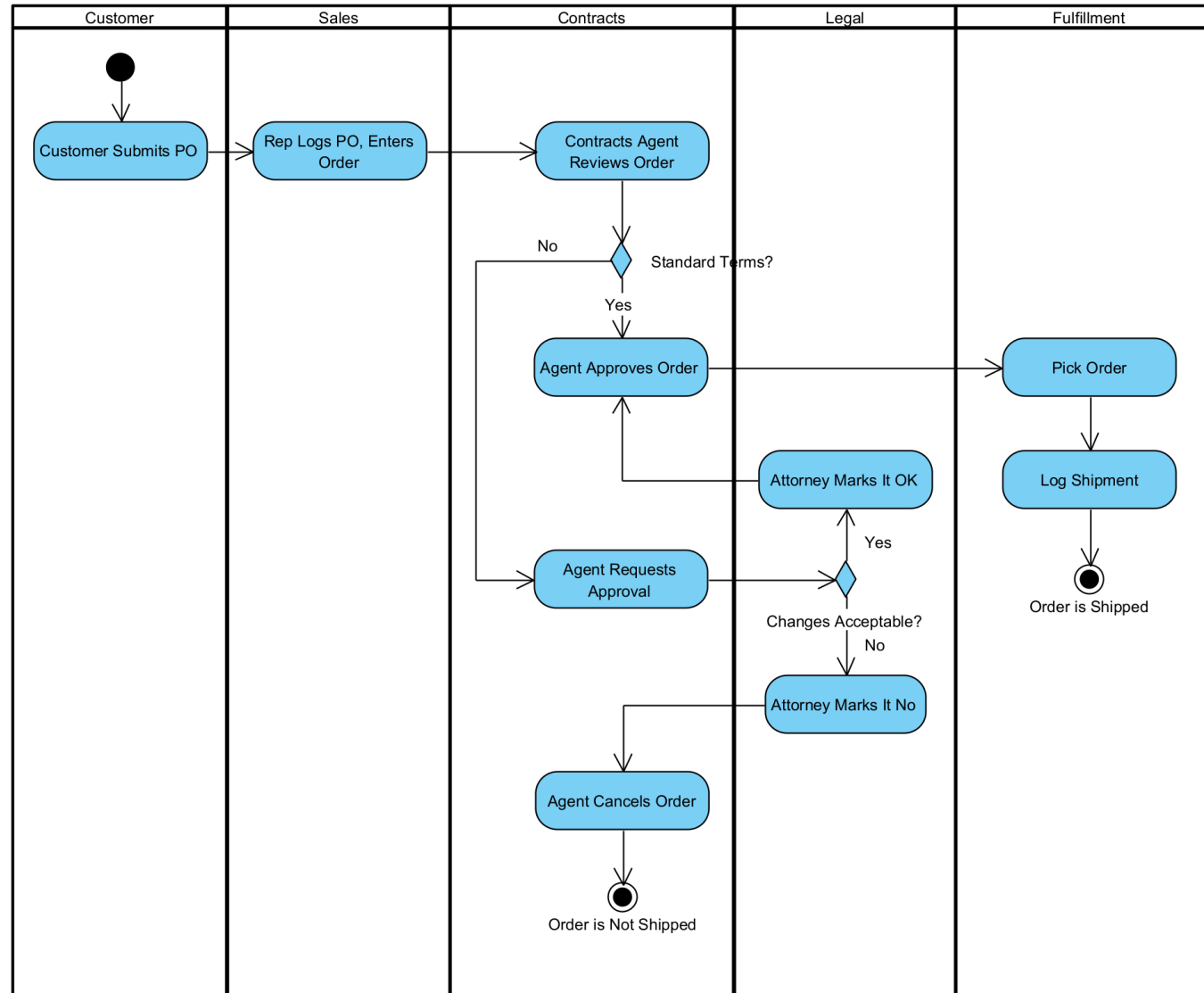


Swimlanes

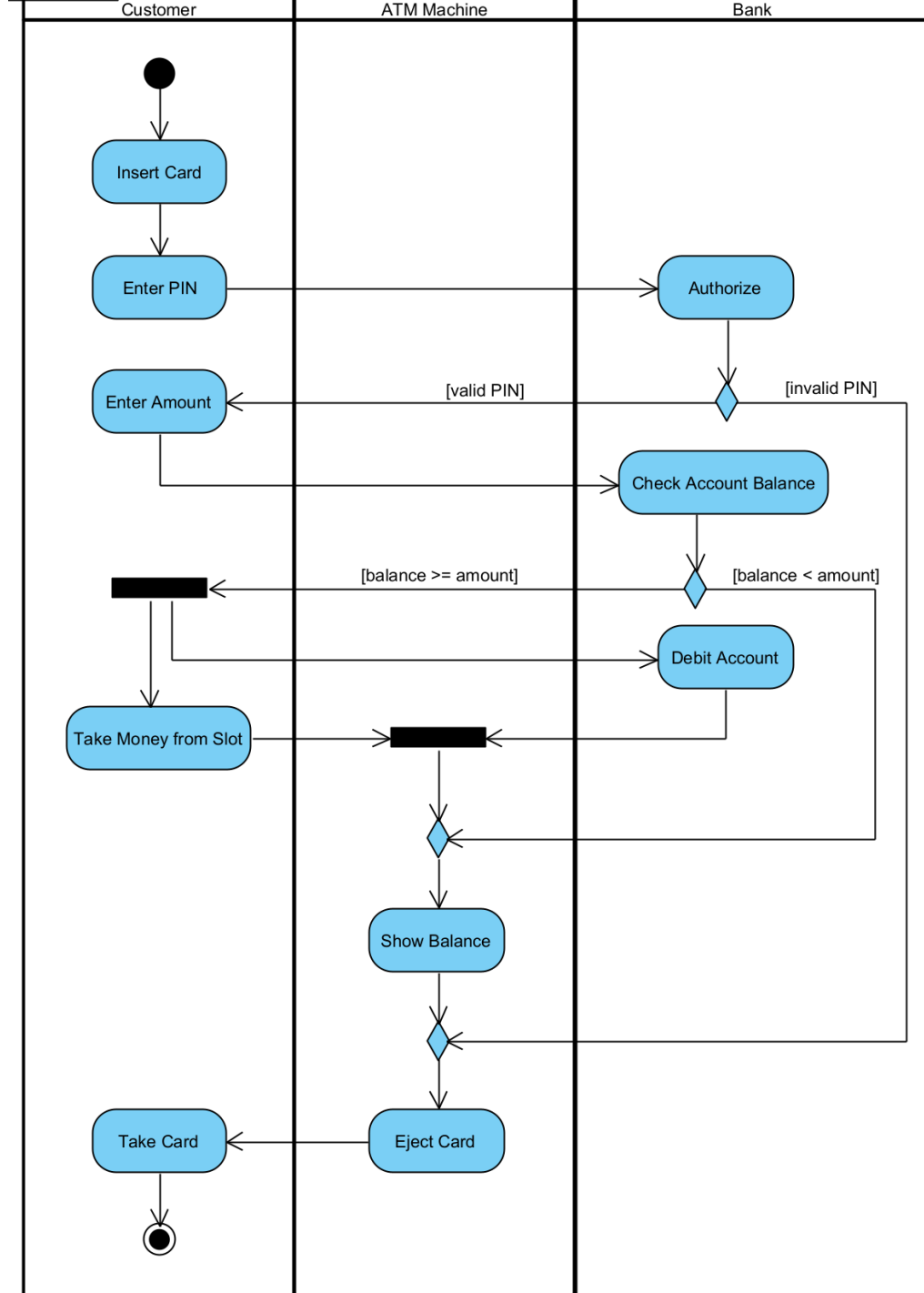
- Activity diagrams can be used to model interactions between different classes
- The activity diagram is divided into vertical 'lanes' (**swimlanes**), in each of which there are the activities that refer to a particular class



Example

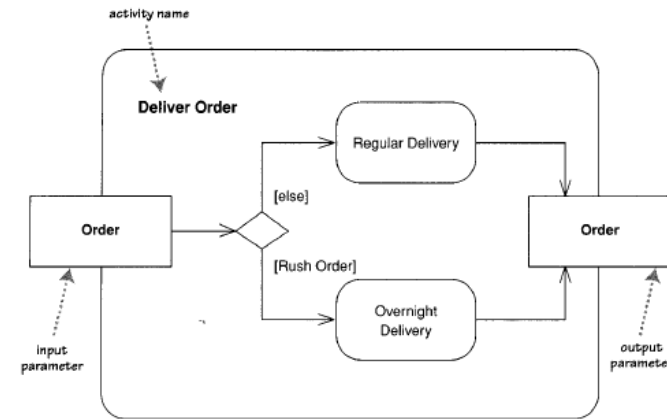
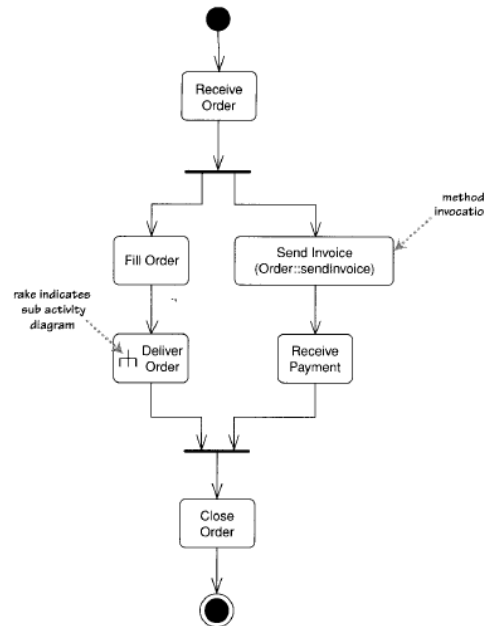


Example



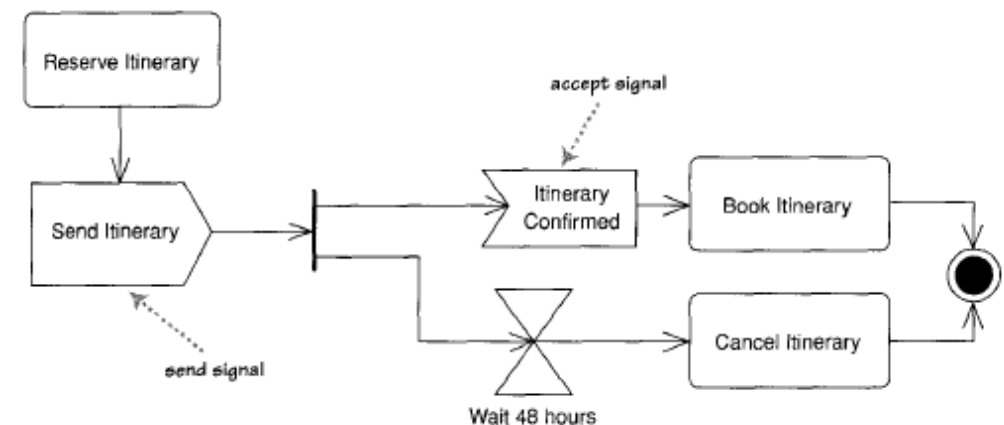
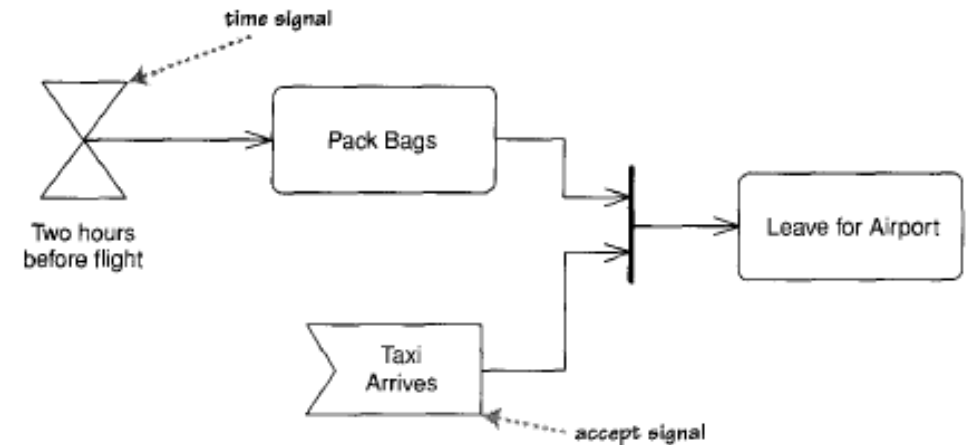
Macro-activity

- The limitation of activity diagrams is probably the overly detailed description of the problems.
- In order to support different levels of abstraction, you can enclose complex tasks in generic tasks, and detail them separately in other charts

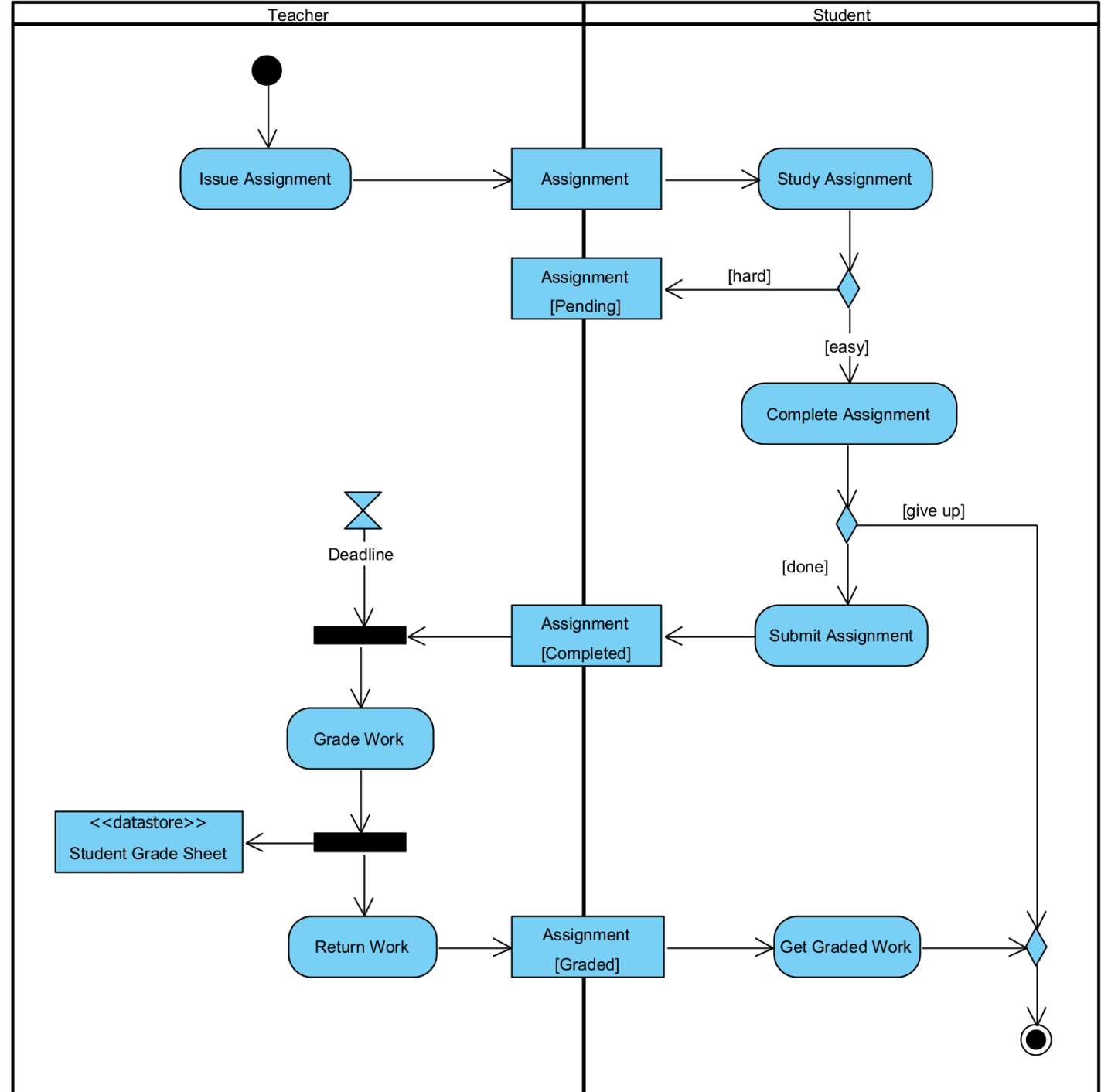


Signals

- Activity diagrams do not explicitly display **events**, unlike statechart diagrams
 - To compensate for this lack, **signals** have been introduced
- A signal indicates any external asynchronous event that affects the execution of an activity
 - A typical case is represented by time signals
- Different symbols indicate
 - Signals sent (send)
 - Received signals
 - Time signals (timers)



Example



References and further readings

- OMG UML Specification (2.5) <https://www.omg.org/spec/UML/2.5/PDF/>
- Ivar Jacobson, James Rumbaugh and Grady Booch. "The unified modeling language reference manual."

